Graphic Era deemed to be University
DEHRADUN

PROJECT-BASED LEARNING

PHASE-I EVALUATION

Project Title:TradeX:A Real Time Stock Order Book and Matching System

Team ID: DSCPP-III-26-2029

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun

Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-26-T029
Semester: 3rd
Section: A,D,F,K
Domain: Btech cse(core)

Team Members

1. *Priyanshi Aggarwal-2510010617*
2. *Aastha Bisht -2510012019*
3. *Khushi Negi-2510012249*
4. *Shivani Mehara-2510012532*

Mentor

Ms Ashwini Yeole

Interactions completed: 02

Problem Statement & Motivation

Problem Statement

Designing a Low Latency Order Matching Engine for Algorithmic Trade Execution.

Motivation

- **Current Gap-** Modern trading systems face delays and inefficiency under heavy data load.
- **Impact on Users-** Users lose profit due to slow matching and poor liquidity.
- **Limitations of existing approaches-** Overcoming issue of heavy databases to guarantee low latency performance.

Target Users / Stakeholders

- **Primary:** Algorithmic traders, fintech developers.
- **Secondary:** Exchanges, researchers, financial institutions.
- **Expected Impact-**Secure, real-time trading with instant matching and cleanup.

Objectives

Project Objectives

1. Develop a secure, high-speed trading engine for real-time order matching.
2. Implement optimized data structures and OOPS principles for instant lookup and cancellation
3. Integrate concepts from **Data Structures & Algorithms** and **OOP**.
4. Evaluate system performance using latency suitable metrics and test cases.
5. Demonstrate practical applicability and future scope.

Key Objective: Achieve microsecond-level trade execution with secure user authentication and efficient memory management.

Proposed Solution

Graphic Era deemed to be University DEHRADUN

Proposed Approach

1. **Input / Data Collection-** Read users.txt for login and orders.txt for bot orders
2. **Processing / Core Algorithm-**Apply Price-Time Priority Cross-Matching and pseudorandom bot generation.
3. **System Logic-**Authenticate users, match trades, and clean completed orders.
4. **Database / Storage-** Use std::map,std::list(linked list) and std::unordered_map for fast access
5. **Output / Visualization-**Display trade alerts and updated order book instantly and graph.

Key Features

- *Feature 1-* Secure user authentication
- *Feature 2-* Real-time order matching
- *Feature 3-* Instant order cancellation in O(1) time
- *Feature 4-* Smart memory management (RAII)
- *Feature 5-* Scalable simulation with 50,000 bot orders

The proposed solution tackles the problem of trading delays by combining efficient data structures with OOP principles. The use of std::map, std::list and std::unordered_map ensures fast lookup , sorting, and cancellation, while encapsulation and RAII provide secure memory handling. Together, these enable real-time trade matching, instant cleanup, and smooth scalability, directly addressing the inefficiency and latency issues in existing systems.

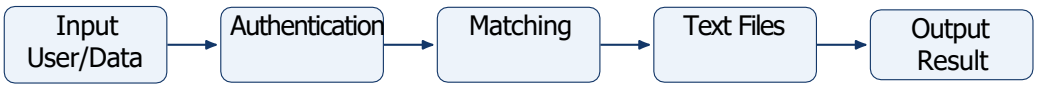
Integration of PBL Course Concepts

Graphic Era deemed to be University DEHRADUN

Course	Concepts Applied	Project Module
Data Structures in C	*Linked Lists, Trees, Hashing, Red-Black Trees(std::map), Doubly Linked Lists(std::list),Hash Index Tables(*std::unordered_map), std::shared_ptr	OrderBook Module(std::map),(std::list), (unordered_map), Order Module(shared_ptr)
OOPs with C++	*Classes, Inheritance, Polymorphism, STL, Encapsulation & Abstraction, Smart Pointers & RAII, Modular Class Hierarchy,File Handling,Authetication*	UserManager module (encapsulation,abstraction),orderbook module(RAII), order.h(inheritance),order class(polymorphism)

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- User Manager- handles login and account creation
- Order book- manages buy/sell queues
- Matching Engine- executes trades and updates states

Data / Control Flow

- **Data Flow**:
  User → Auth →Order → Match → Confirm →Cleanup
- **Modules Interaction**:
  User Manager authenticates → Order Book queues → Matching Engine executes → DB log → Output displays results
- **Expected Output**:
  Real-time trade confirmations, updated order book, and Secure sessions, optimized memory

Technology Stack & Methodology

Technology Stack

- Programming: *C / C++*
- Database: *Text Files*
- Tools / Framework: C++(STL)
- IDE: *Visual Studio*
- Version Control: *Git / GitHub*

Development Methodology

1. Requirement Analysis- trading efficiency, low-latency needs
2. System Design- modular C and C++ architecture, Text Files.
3. Implementation- OOPS and DSA with STL.
4. Testing- simulated trades, latency & security
5. Evaluation- memory usage, user experience

The selected technologies ensure **speed, reliability, and scalability** — essential for a high-frequency trading engine. C++ and STL provide high yield performance, text files guarantee data maintenance and GitHub supports smooth teamwork and continuous progression.

Initial Progress & Team Contribution

Progress Achieved

- *Review of research papers on stocks and trading systems*
- *Identification of system requirements and modularity*
- *Architecture and workflow design with prototype setup in C++ using STL*
- *Created GitHub repository for future development*

Individual Contribution

Member	Role	Contribution
M1 (Priyanshi)	*Team Lead and System Architect*	System architecture,module planning,integration, technical coordination and core engine
M2 (Aastha)	*Core Engine Developer*	Matching algorithms,order-processing logic,core engine,optimization and integration.
M3 (Khushi)	Application Developer	Frontend and backend modules, API integration and DB connectivity.
M4 (Shivani)	Data and QA Engineer	*Data Management, testing,validation, debugging and documentation*

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
   - Problem Identification, references and planning of design
2. Phase-II: Development
   - Implementation of modules , algorithms and testing and integration of system
3. Phase-III: Final Implementation
   - System evaluation and optimization .

Expected Outcomes

- Working real-time trading prototype
- Verified low-latency performance
- Practical applicability
- Scalable and secure system design

References

1) R.Cont,S.Stoikov,andR.Talreja, "Astochasticmodel fororderbookdynamics,"OperationsResearch, vol. 58,no. 3, pp. 549–563,2010.

2) L.Gai andS.Yao, "Architectingultra-low latencyplatforms via in-memory systems," IEEETransactions onServices Computing,vol. 16,no. 2,pp. 1142–1155,2023.

3) X.Zhang,H.Yang,andJ.Liu,"High-throughputkey-valuelookuparchitecturesforfinancial systems,"ACMTransactions onEmbeddedSystems,vol. 21,no. 4,pp. 42–58,2022.

Thank You
Questions & Suggestions